



WEEK 4

SOFTWARE DEVELOPMENT TOOLS AND ENVIRONMENTS



Presented by Asst. Prof. Dr. Tuchsana Ploysuwan



Week 4

Undoing Changes

Week 4 Undoing Changes

- Today we're focused on how to undo actions related to git commands and explore historical commits.
- We'll discuss:
 - **git checkout** and **Detached HEAD**
 - **git restore**
 - **git reset**
 - **git revert**

Week 4

Git Checkout

Week 4 Undoing Changes

- **git checkout**

- This is actually a very versatile command, so versatile in fact, that developers complained it was used for too many different actions, thus new git commands were created, such as **git switch**.

Week 4 Undoing Changes

- **git checkout**

- A "checkout" is the act of switching between different versions of a target entity.
- The **git checkout** command can operate on three distinct entities: files, commits, and branches.

Week 4 Undoing Changes

- **git checkout**

- For example, you could use **git checkout branch_name** instead of **git switch branch_name** to checkout a new branch.
- Unlike **git switch** however, recall checkout can operate on commits, meaning we can “checkout” historical commits.

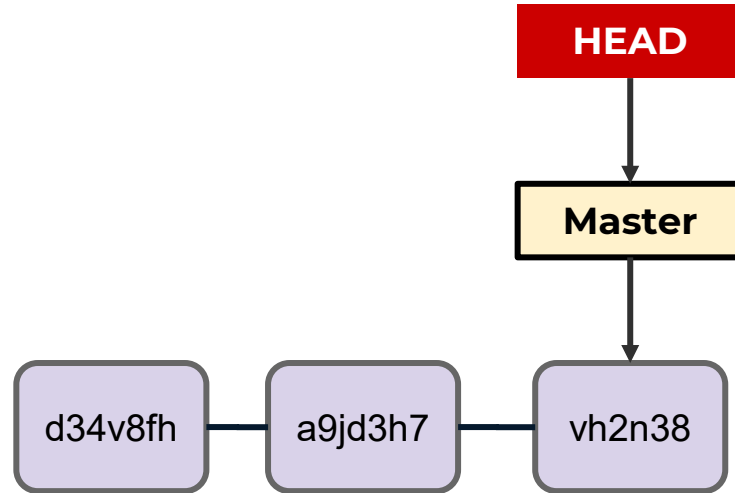
Week 4 Undoing Changes

- **git checkout**

- We can check out a particular commit by specifying its hash, we can get hashes from the **git log** command and we can also see the abbreviated hash using:
 - **git log --oneline**
- Then we can provide the has as:
 - **git checkout #####**

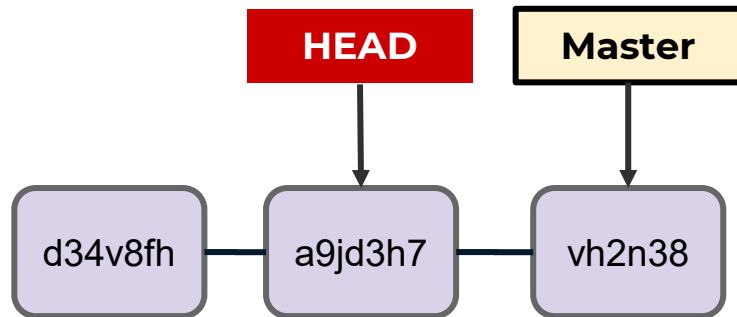
Week 4 Undoing Changes

- Typically our HEAD points to the branch which points to the latest commit.



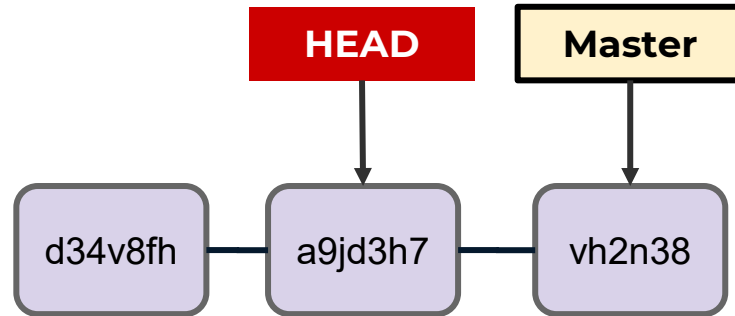
Week 4 Undoing Changes

- Upon calling **git checkout a9jd3h7** we detach the HEAD to a previous commit



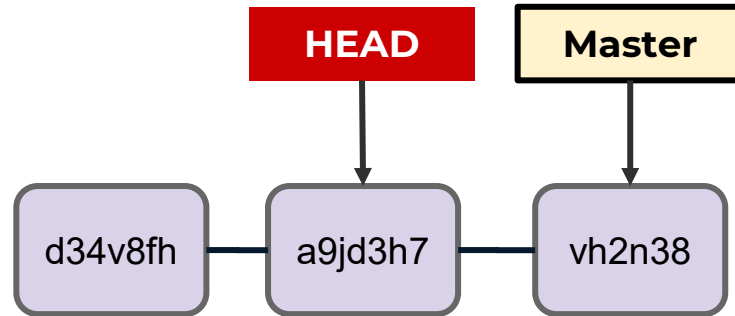
Week 4 Undoing Changes

- You can think of this as traveling back in history to what your code looked like when you ran this commit.



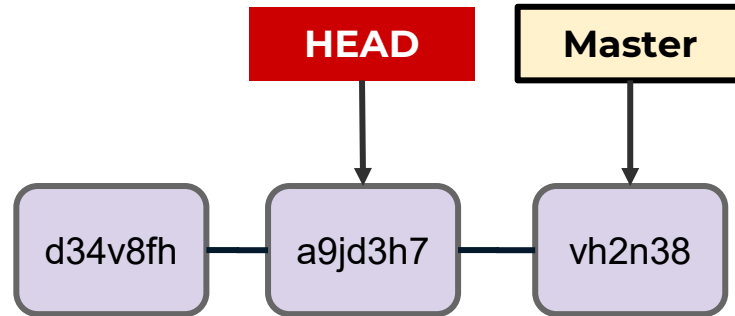
Week 4 Undoing Changes

- This command does **not** undo previous work, you are simply exploring the historical commit.



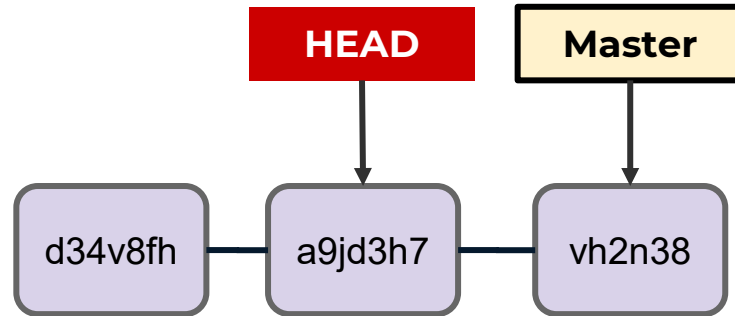
Week 4 Undoing Changes

- If you started making changes here, they won't be preserved since HEAD is not pointing at a branch reference.



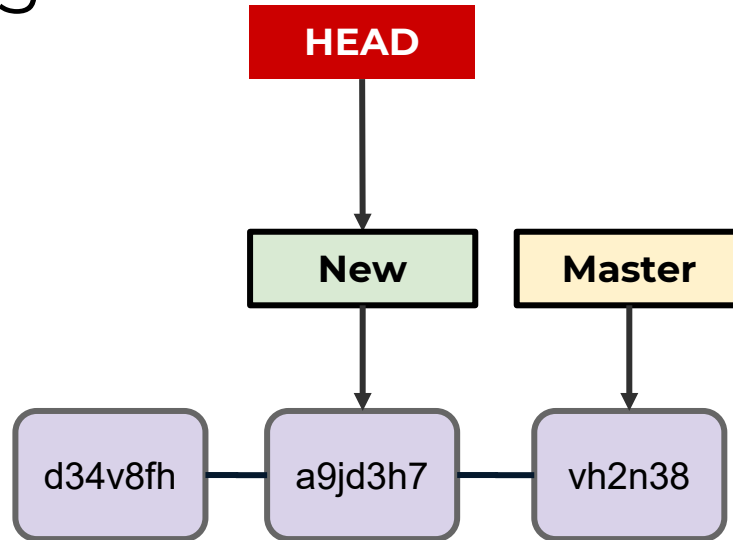
Week 4 Undoing Changes

- However you could create a new branch at this point in time, reattaching HEAD to a branch again.



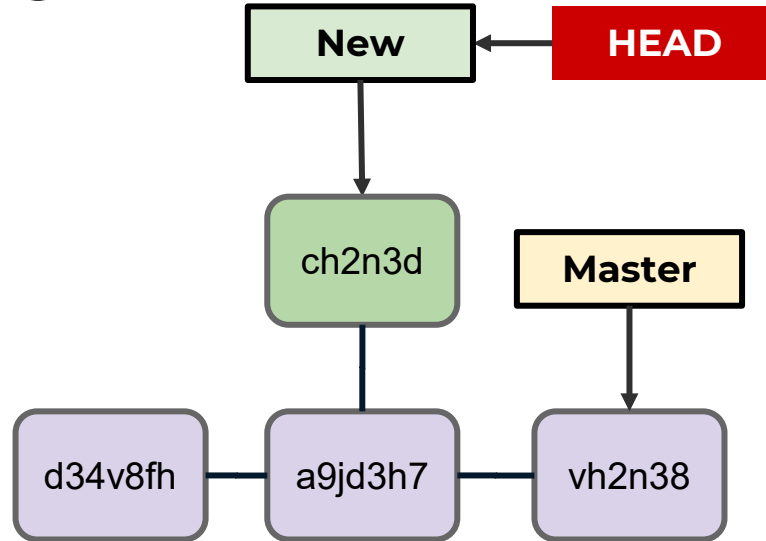
Week 4 Undoing Changes

- However you could create a new branch at this point in time, reattaching HEAD to a branch again.



Week 4 Undoing Changes

- However you could create a new branch at this point in time, reattaching HEAD to a branch again.



Week 4 Undoing Changes

- Let's explore these commands in practice!

Week 4

Git Restore

Week 4 Undoing Changes

- Imagine you've been working on a file that is already part of a commit.
- You know the code was working at the point of the last commit, however now the code is totally broken, and you just can't seem to fix it.
- You've written too much code to just run Ctrl+Z! How to fix this?

Week 4 Undoing Changes

- **Git restore**

- We can restore a file to its state at the previous most recent commit using the

git restore command:

- **git restore file_name**

Week 4 Undoing Changes

- **Git restore**

- We can restore a file to its state at the previous most recent commit using the

git restore command:

- **git restore file_name**

- ***Warning:***

- You can not undo a git restore command, since your changes were not committed!

Week 4 Undoing Changes

- **Git restore**

- We can restore a file to its state at the previous most recent commit using the **git restore command**:
 - **git restore file_name**
- **Warning:**
 - Think of this command as an ultimate “Ctrl+Z” restoring files to their previous commit.

Week 4 Undoing Changes

- **Git restore**

- Technically speaking **git restore** will restore the file back to the HEAD, which typically we have pointing to the most recent commit in the branch.

Week 4 Undoing Changes

● Git restore

- This actually gives us even more flexibility in our restore procedure, we can restore a file to any commit in the log.
- We state the number of commits from the HEAD to go back to:
 - **git restore --source HEAD~N file.txt**

Week 4 Undoing Changes

● Git restore

- This actually gives us even more flexibility in our restore procedure, we can restore a file to any commit in the log.
- We state the number of commits prior from the HEAD to go back to:
 - **git restore --source HEAD~N file.txt**

Week 4 Undoing Changes

● Git restore

- This actually gives us even more flexibility in our restore procedure, we can restore a file to any commit in the log.
- We state the number of commits prior from the HEAD to go back to:
 - **git restore --source HEAD~N file.txt**

Week 4 Undoing Changes

- **Git restore**

- Finally, git restore also allows us to unstage files that we had already added to the staging area using **git add**.
- We can do this with:
 - **git restore --staged filename**

Week 4 Undoing Changes

- Let's explore this command in practice:
 - **git restore filename**
 - **git restore --source HEAD~N filename**
 - **git restore --staged filename**

Week 4

Git Reset

Week 4 Undoing Changes

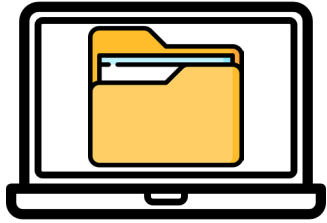
- Git reset allows us to remove commits and “reset” the branch.
- There are two main types of **git reset** calls:
 - **git reset #####**
 - Removes commits in front of the specific hash called, files unchanged.
 - **git reset ##### --hard**
 - Removes commits *and* the changes in the files.

Week 4 Undoing Changes

- To fully understand this, let's recall our discussions about working directory, staging area, and repository.

Week 4 Undoing Changes

Working Directory



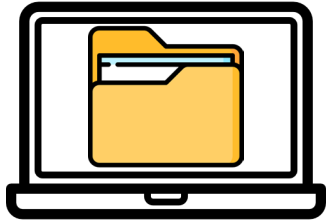
program.py

Staging Area

Repository

Week 4 Undoing Changes

Working Directory



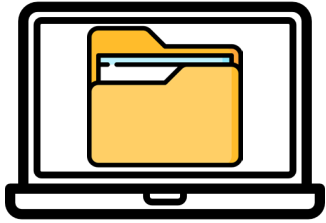
Staging Area

program.py

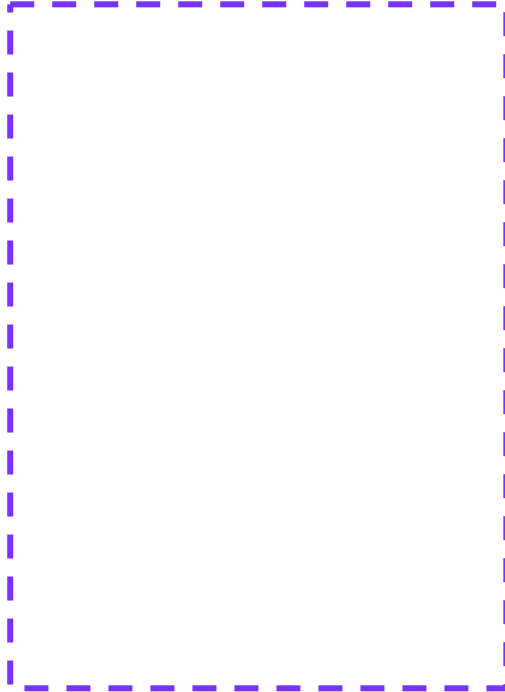
Repository

Week 4 Undoing Changes

Working Directory



Staging Area



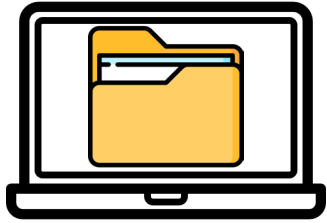
Repository

`program.py`

“python code”
f3h4782

Week 4 Undoing Changes

Working Directory



index.html

style.css

Staging Area

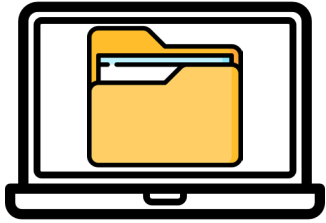
Repository

program.py

“python code”
f3h4782

Week 4 Undoing Changes

Working Directory



Staging Area

index.html

style.css

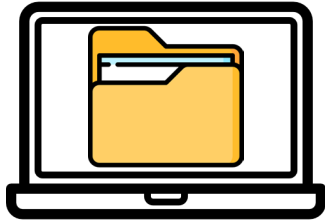
Repository

program.py

“python code”
f3h4782

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

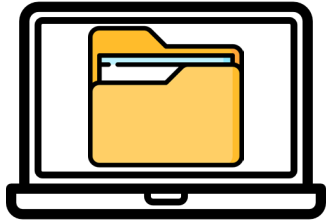
index.html

style.css

“style code”
g82j37l

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

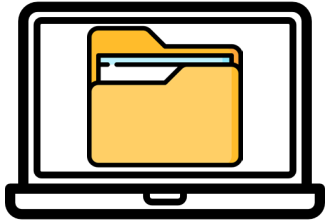
index.html

style.css

“style code”
g82j37l

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

index.html

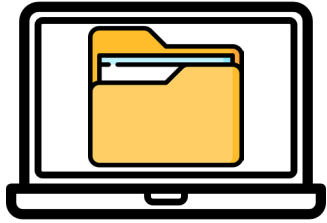
style.css

“style code”
g82j371

```
>>git reset f3h4782
```

Week 4 Undoing Changes

Working Directory



index.html

style.css

Staging Area

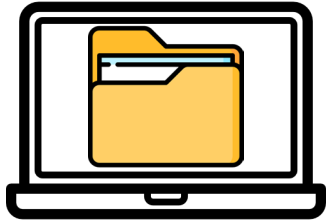
Repository

program.py

“python code”
f3h4782

Week 4 Undoing Changes

Working Directory



index.html

style.css

Staging Area

Files are unchanged!
You just reset the
commits only.

Repository

program.py

“python code”
f3h4782

Week 4 Undoing Changes

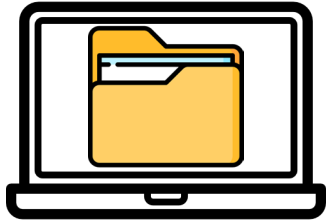
- This means when you do **git reset** you won't notice a change in the files themselves, you just reset the commits.
- This is useful if you accidentally committed to the wrong branch (for example, maybe you forgot to run **git switch** right after creating a new branch, accidentally committing to the original branch).

Week 4 Undoing Changes

- What if you do want the files to change?
- In the case where you just want to undo everything, including changes and have the branch files look like they did at a previous commit, you add the flag **--hard**.
- For example:
 - **git reset f3h4782 --hard**

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

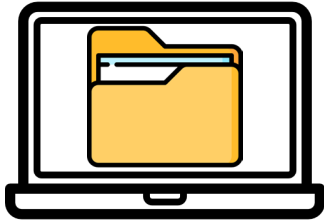
index.html

style.css

“style code”
g82j371

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

index.html

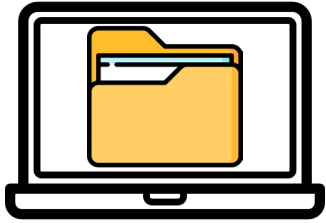
style.css

“style code”
g82j371

```
>>git reset f3h4782 --hard
```

Week 4 Undoing Changes

Working Directory



Staging Area

Repository

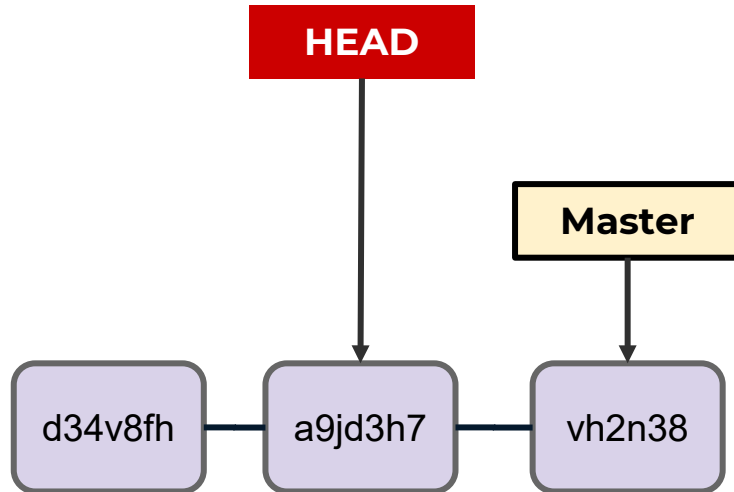
program.py

“python code”
f3h4782

```
>>git reset f3h4782 --hard
```

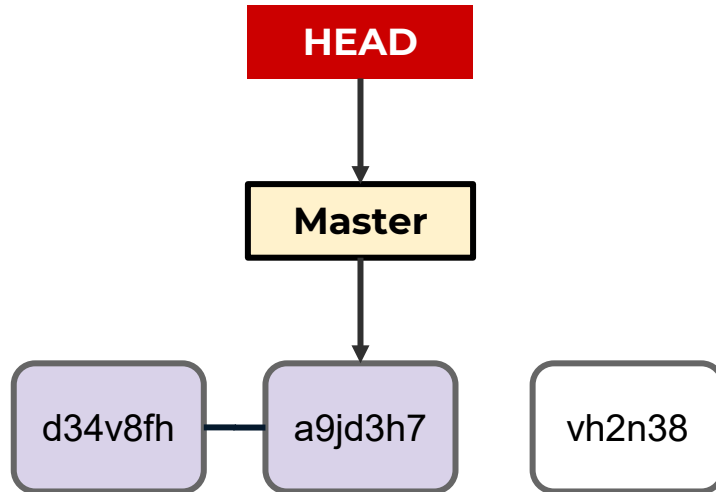
Week 4 Undoing Changes

- We can visualize a **git reset** moving back to a previous commit, but not undoing file changes (unless it is --hard)



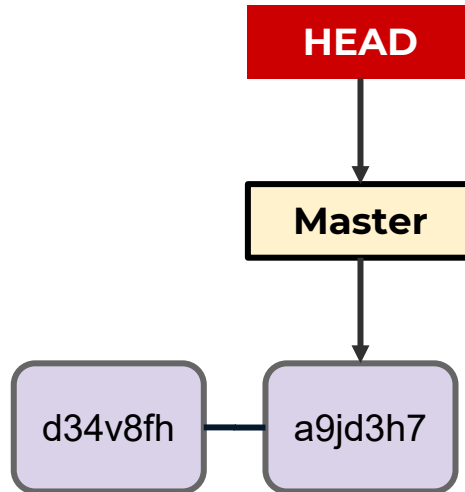
Week 4 Undoing Changes

- We can visualize a **git reset** moving back to a previous commit, but not undoing file changes (unless it is --hard)



Week 4 Undoing Changes

- We can visualize a **git reset** moving back to a previous commit, but not undoing file changes (unless it is --hard)



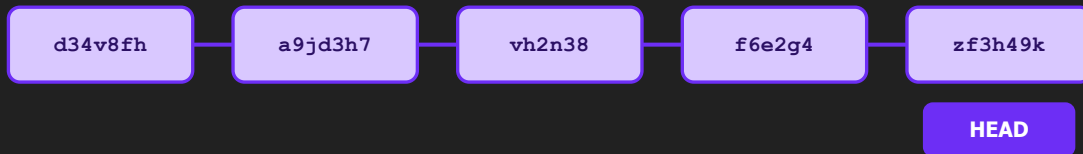
Week 4

Git Revert

ทบทวน: git reset ทำอะไร

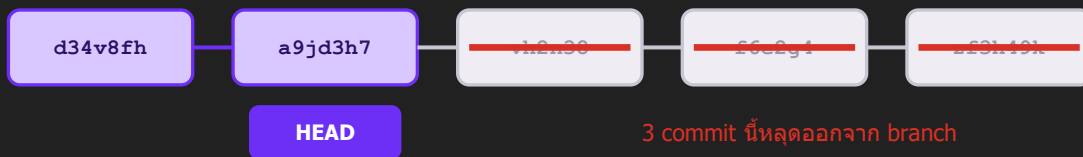
ย้าย branch pointer ถอยหลัง → commit ที่อยู่ข้างหน้าหลุดออกจากประวัติ

ก่อน reset



```
>> git reset a9jd3h7 --hard
```

หลัง reset

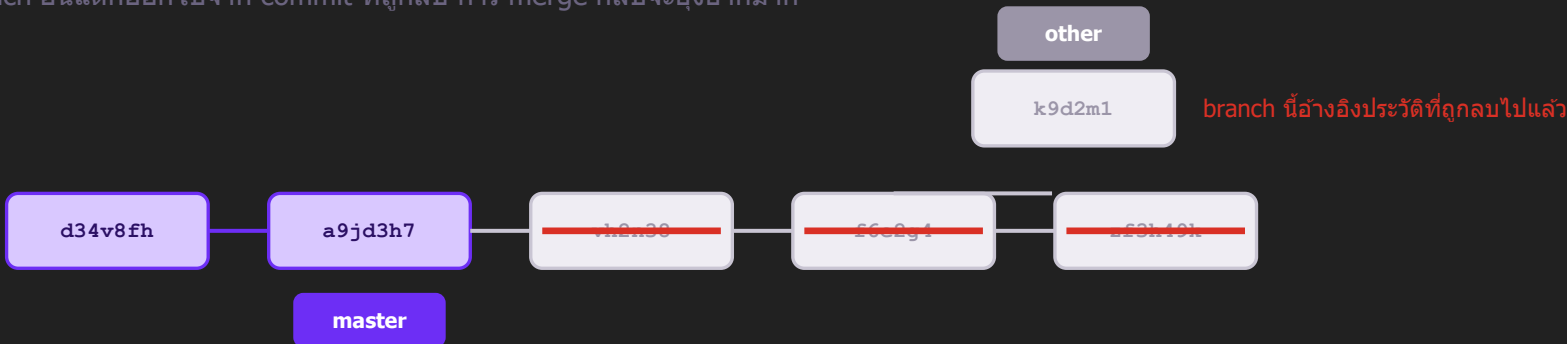


ผลที่ตามมา

- ประวัติถูกเขียนใหม่ (rewrite history)
- เพื่อนที่ pull ไปแล้ว มีประวัติไม่ตรงกับเรา
- ต้องใช้ git push --force → เสี่ยงลบงานคนอื่น
- --hard ให้ถือว่า กู้คืนไม่ได้

ทำไม reset ถึงอันตราย: เสีย shared history

ถ้ามี branch อื่นแตกออกไปจาก commit ที่ถูกลบ การ merge กลับจะยุ่งยากมาก



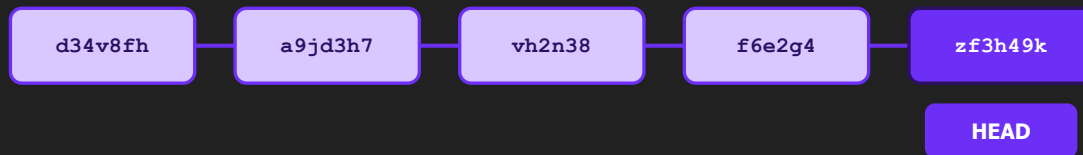
หลักคิด

git reset = “แกล้งทำเป็นว่าไม่เคยเกิดขึ้น” → ใช้ได้เฉพาะ **commit** ที่ยังไม่เคย **push**

git revert คืออะไร

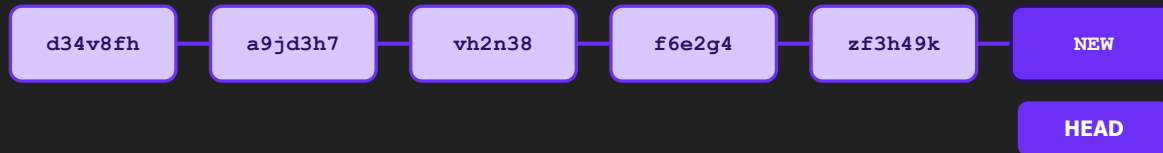
forward-moving undo — สร้าง commit ใหม่ที่มีเนื้อหา “กลับด้าน” แทนการลบ commit เดิม

ก่อน revert



```
>> git revert zf3h49k
```

หลัง revert



commit เดิมอยู่ครบทุกตัว + commit ใหม่ที่ลบล้างผลของ zf3h49k



ข้อดี

- ประวัติไม่ถูกเขียนใหม่
- push ได้เลย ไม่ต้อง --force
- เพื่อนร่วมทีมแค่ git pull ปกติ
- ตรวจสอบย้อนหลังได้ว่าใครยกเลิกอะไร

เปรียบเทียบให้เห็นภาพ: การทำบัญชี



git reset = ฉีกหน้าบัญชีทั้ง

- รายการที่ผิดพลาดหายไปเลย
- สมุดบัญชีดูสะอาด
- แต่ผู้ตรวจสอบไม่รู้ว่าจะเกิดอะไรขึ้น
- ถ้ามีคนถ่ายสำเนาไปแล้ว → เล่มไม่ตรงกัน



git revert = เขียนรายการปรับปรุงยอด

- รายการเดิมยังอยู่
- เพิ่มรายการใหม่ที่หักล้างยอดออก
- ยอดสุทธิถูกต้องเหมือนกัน
- ตรวจสอบย้อนหลังได้ครบ (auditable)

สรุปเปรียบเทียบ 4 คำสั่งของ Week 4

คำสั่ง	ทำอะไร	ประวัติเปลี่ยน?	ปลอดภัยกับ shared repo
<code>git checkout <hash></code>	ไปดูโค้ดในอดีต (detached HEAD)	ไม่	ปลอดภัย (แค่ดู)
<code>git restore <file></code>	ดึงไฟล์กลับเป็นเวอร์ชันที่ commit ไว้	ไม่	ปลอดภัย
<code>git reset <hash></code>	ย้าย pointer ถอยหลัง ลบ commit	ใช่ — เขียนใหม่	อันตราย
<code>git revert <hash></code>	สร้าง commit ใหม่ที่ย้อนผล	ไม่ (เพิ่ม commit)	ปลอดภัยที่สุด

checkout / restore = ปลอดภัยเสมอ • **reset** = เขียนประวัติใหม่ (ระวัง) • **revert** = ตัวเลือกเดียวที่ทำได้กับ commit ที่ push แล้ว

ตัวอย่างที่ 1 — เตรียม repo ทดลอง

สร้าง 3 commit โดย commit สุดท้ายคือ commit ที่ “ผิดพลาด”

```
mkdir revert-demo && cd revert-demo
git init
```

```
echo "print('hello')" > app.py
git add app.py
git commit -m "C1: add app.py"
```

```
echo "print('feature A')" >> app.py
git commit -am "C2: add feature A"
```

```
echo "print('DEBUG')" >> app.py
git commit -am "C3: add debug code"
```

```
git log --oneline
```

```
a3f9c21 C3: add debug code
7b21e04 C2: add feature A
1c8d3f5 C1: add app.py
```

app.py มี 3 บรรทัด — บรรทัดสุดท้ายคือโค้ด debug

ตัวอย่างที่ 1 — สั่ง revert แล้วดูผล

Git จะเปิด editor ให้ยืนยัน commit message ที่เตรียมไว้ให้ — บันทึกแล้วปิดได้เลย

```
>> git revert a3f9c21
```

ประวัติหลัง revert

```
d55b8a9 Revert "C3: add debug code"
a3f9c21 C3: add debug code      <- ยังอยู่!
7b21e04 C2: add feature A
1c8d3f5 C1: add app.py
```

ไฟล์ app.py หลัง revert

```
print('hello')
print('feature A')
print('DEBUG')
```

บรรทัด debug หายไปจากไฟล์แล้ว แต่ commit a3f9c21 ยังอยู่ในประวัติครบถ้วน